

Vision Day4 HW1 레포트

로봇 예비단원 주호진

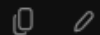
1. AI 보고서

<https://share.gemini.google/adSM6SLowmcr>

<https://share.gemini.google/L0bXgQNN2GMH>

<https://share.gemini.google/qdhpHiM8XT2F>

pytorch는 파이썬으로 구현하는게 제일 나은가



결론부터 말씀드리면, 파이썬(Python)으로 구현하는 것이 가장 좋고, 사실상 표준이자 필수입니다.

요약 이랜서

특별한 임베디드 환경 배포나 시스템 제약이 있는 경우가 아니라면, 모델 학습, 파인튜닝, 연구 등 모든 인공지능 개발 작업은 파이썬과 파이토치를 조합하는 것이 가장 효율적이고 검증된 방법입니다.

파이썬이랑 cpp 중 아무거나 선택해서 구현해도 상관없어서 뭘로 구현할지 물어봤다. 이름부터 pytorch지만 여태까지 과제를 cpp로 진행해서 혹시나 해서 물어봤는데, 거의 반 필수로 파이썬을 쓴다고 한다. 그래서 카메라 노드만 cpp로 만들었다.

패키지가 2개인데 하나의 런치파일로 실행할 수 있어?

결론부터 말씀드리면, 네, 당연히 가능합니다!

ROS 2 런치 파일의 가장 큰 장점 중 하나가 바로 서로 다른 패키지(C++ 패키지든, 파이썬 패키지든)에 흩어져 있는 노드들을 하나의 파일로 묶어서 동시에 실행할 수 있다는 점입니다.

💡 실무에서 자주 쓰는 팁: `bringup` 패키지 분리하기

C++ 패키지 안에 런치 파일을 넣어도 되고, 파이썬 패키지 안에 넣어도 되지만, 프로젝트가 커지면 보통 런치 파일과 설정 파일만 모아두는 빈 패키지(예: `robot_bringup` 등)를 따로 하나 더 만듭니다.

전체 구조가 이렇게 구성됩니다.

Plaintext

```
ros_ws/src/
├─ cpp_control_pkg/      # C++ 노드 패키지
├─ pytorch_ai_pkg/       # 파이썬 PyTorch 노드 패키지
└─ robot_bringup/        # 런치 파일만 모아둔 패키지 (선택사항이지만 추천)
    └─ launch/
        └─ total_system.launch.py
```

cpp 노드, 파이썬 노드가 있어서 패키지가 2 개가 되어서 런치파일을 어떻게 해야하는지 물어봤다. 이런 경우 런치파일만 있는 빈 패키지를 만들어서 쓴다고 하길래 해당 구조를 채택했다.

ros2 패키지 상에서 학습된 모델 파일은 어느 폴더 구조에 들어가는 게 일반적이야?

ROS 2 패키지에서 학습된 모델 파일(`.pt` , `.onnx` , `.engine` 등)은 보통 패키지 루트 디렉터리의 `config/` 또는 `weights/` (혹은 `models/`)폴더에 넣는 것이 일반적입니다.

모델 파일을 패키지에 넣어야 하는데 어떤 폴더 하위에 두는지 물어보았다. Config, weights, models 를 쓴다고 했는데, config 에는 yaml 파일이 들어가므로 weights, models 중에 models 가 이름이 더 직관적으로 느껴져서 models 폴더 아래에 넣었다.

Sets the Intersection over Union threshold used by Non-Maximum Suppression to remove overlapping detections. Lower values suppress more overlapping boxes; higher values keep more nearby boxes. Valid range: 0.00-0.95. In Python, this option is `iou`.

이거 뭘소리야

한 마디로 "똑같은 물체에 박스가 여러 개 겹쳐서 쳐졌을 때, 어느 정도 겹쳐야 중복으로 판단하고 지울 것인가"를 정하는 기준값입니다.

코코 데이터셋 yolo26n predict 탭에 사이트 상에서 테스트 해볼 수 있는 게 있길래 보고 있었는데, 하단에 conf, IoU, image size 가 있길래 각각 파파고로 설명을 보는데 IoU 는 파파고 해석을 봐도 무슨 소리인지 모르겠어서 설명해달라고 했다. Conf 는 ppt 에 나와있고, image size 는 파파고 해석으로 봐도 이해가 잘 되어서 iou 만 질문했다.

conf

탐지 최소 confidence score

해당 값이 낮아지면 탐지율이 높아지지만 오탐지율도 같이 높아짐

해당 값이 높아지면 탐지율은 떨어지지만 오탐지율 줄어듦

range: 0.01 ~ 1.00 / site default = 0.25

iou

예측한 두 박스가 겹치는 영역의 비율(1: 완전 겹침, 0: 완전히 겹치지 않음)

NMS (Non-Maximum Suppression): AI 가 하나의 object 에 대한 박스를 여러개 겹쳐서 그렸을 때, 가장 정확한 1 개만 남기고 나머지를 지워주는 후처리 기술

낮으면 조금만 겹쳐도 지움(confidence 가 가장 높은 박스가 살아남음)

높을수록 점점 안지움

range: 0.00 ~ 0.95 / site default = 0.70

imagsz

해상도 설정

높을수록 정확도 높아지지만 지연 시간 증가

range: 32 ~ 1280 / site default = 640

qos depth가 1인데 어쩌다 한번 지연되면 지연시간이 계속해서 지속되는데 이유가 뭐야

QoS의 `depth` 를 1로 설정하면 이론적으로는 "가장 최신 메시지 1개만 남기고 나머지는 다 버리기 때문에" 지연이 누적되지 않고 실시간성을 유지해야 합니다.

그럼에도 불구하고 어쩌다 한 번 생긴 지연이 풀리지 않고 계속 지속되거나 심해지는 이유는 ROS의 통신 계층 외적인 다른 곳에서 병목과 큐 적체가 발생하고 있기 때문입니다. 대표적인 원인 3가지는 다음과 같습니다.

1. OpenCV GUI 창(`cv.imshow`)과 이벤트 루프의 밀림 (가장 흔한 원인)
2. PyTorch / CUDA의 비동기(Asynchronous) 연산 큐 적체

3. ROS 콜백 함수 내부의 무거운 연산 (CPU/GIL 병목)

해결 방법

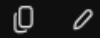
이 문제를 해결하려면 ROS 콜백 함수 안에서는 무거운 작업을 직접 다 하지 말고, 데이터만 가볍게 받아온 뒤 별도의 스레드나 메인 루프에서 처리해야 합니다.

가장 추천하는 구조는 다음과 같습니다: [ROS Answers](#)

1. 콜백 함수: 카메라도착 메시지(`msg`)를 전역 변수나 클래스 멤버 변수(`self.latest_image`)에 저장만 하고 즉시 리턴합니다. (지연 방지) [ROS documentation](#)
2. 별도 루프 (또는 타이머): `while` 문이나 ROS 타이머를 이용해 최신 이미지를 가져와서 YOLO 추론과 `cv.imshow` 를 수행합니다. 이 방식을 쓰면 GUI나 추론이 아무리 지연되어도 이전 잔여 이벤트나 큐가 꼬이지 않고 항상 최신 상태를 유지할 수 있습니다.

화면에 대고 팔을 빠르게 휘적거리면 프레임이 엄청 떨어져서 질문하게 됐다. 문제가 될 수 있는 부분을 3 가지 제시해줬고, 해결 방법으로 콜백 함수에 모든 코드가 들어있으면 무거울 수 있으니 따로 분리하라고 해서 일단 sub 콜백에서는 이미지만 받아오고 나머지 코드를 타이머 콜백으로 분리했다.

타이머 콜백 분리했는데 지연이 생기면 밀리는데



타이머나 콜백을 분리했음에도 불구하고 지연이 발생할 때 점점 더 밀리는(누적되는) 현상이 생긴다면, "이미지를 처리하는 속도(AI 추론 시간)"가 "카메라가 이미지를 보내는 속도(프레임 레이트)"보다 느리기 때문입니다.

```
# 최신 이미지를 담을 변수와 동기화를 위한 락(Lock)
self.latest_image = None
self.lock = threading.Lock()
```

lock이 mutex같은건가

네, 정확히 맞습니다!

Python의 `threading.Lock()` 은 운영체제나 다른 프로그래밍 언어에서 부르는 뮤텍스(Mutex, Mutual Exclusion)와 완전히 같은 개념입니다.

타이머 콜백으로 따로 빼서 분리해줬는데 똑같이 딜레이가 걸리길래 다시 질문했다. `threading.lock()`을 사용해보라는 답변이 있었고, Mutex 처리를 하니까 딜레이가 사라졌다. Sub 콜백에서 받아온 이미지를 타이머 콜백에서 참조하는데 타이머 콜백에서 코드가 작동하는 도중에 이미지가 sub 콜백에서 바뀌게 되면서 메모리 충돌이 일어나 지연이 된 게 문제였다.